

Lecture 12

Greedy Algorithms



Slides are based on the textbook and its notes

Overview

- ❑ Similar to dynamic programming.
- ❑ Used for optimization problems, simpler, more efficient than dynamic programming.
- ❑ *Idea*
 - A **greedy algorithm** always makes the choice that looks best *right now*. Make a *locally optimal choice* in hope of getting a *globally optimal solution*.
 - Greedy algorithms do not always yield an optimal solution. But for many problems they do.
 - We will study one application of the greedy method: Huffman coding.

Huffman codes

- **Goal:** Compress a data file made up of characters.
- You know how often each character appears in the file – its *frequency*.
- Here we consider the problem of designing a *binary character code* (or *code* for short), in which each character is represented by a unique binary string: a *codeword*. Use as few bits as possible to represent the file.
- **Fixed-length code:** All codewords have the same number of bits. For $n \geq 2$ characters, need $\lceil \lg n \rceil$ bits.
- **Variable-length code:** Represent different characters with differing numbers of bits. In particular, give frequently occurring characters shorter codewords and infrequently occurring characters longer codewords.

Huffman codes

- *Example:* For a data file of 100,000 characters:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

- For a fixed-length code, need 3 bits per character. For 100,000 characters, need 300,000 bits. For this variable-length code, need

$$\begin{array}{rcl} 45,000 \cdot 1 & = & 45,000 \\ + 13,000 \cdot 3 & = & 39,000 \\ + 12,000 \cdot 3 & = & 36,000 \\ + 16,000 \cdot 3 & = & 48,000 \\ + 9,000 \cdot 4 & = & 36,000 \\ + 5,000 \cdot 4 & = & 20,000 \\ \hline & = & 224,000 \text{ bits} \end{array}$$

Huffman codes

□ *Prefix-free codes*

No codeword is also a prefix of any other codeword. A prefix-free code can always achieve the optimal compression.

□ *Encoding*: Just concatenate codewords for each character in the file.

Example: To encode `face`: $1100 \cdot 0 \cdot 100 \cdot 1101 = 110001001101$, where $\cdot$ is concatenation.

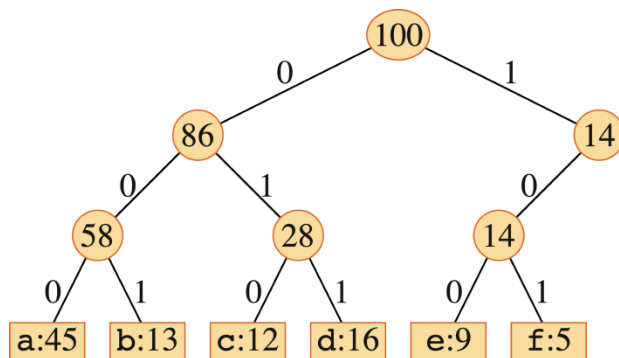
□ *Decoding*: Since no codeword is a prefix of any other codeword, just process bits until you get a match. Then discard the bits and go from the rest of the compressed file.

Example: If encoding is `100011001101`, get a match on `100` = `c`. That leaves `011001101`. Get a match on `0` = `a`. That leaves `11001101`. Get a match on `1100` = `f`. That leaves `1101`. Get a match on `1101` = `e`. So the encoded file represents `cafe`.

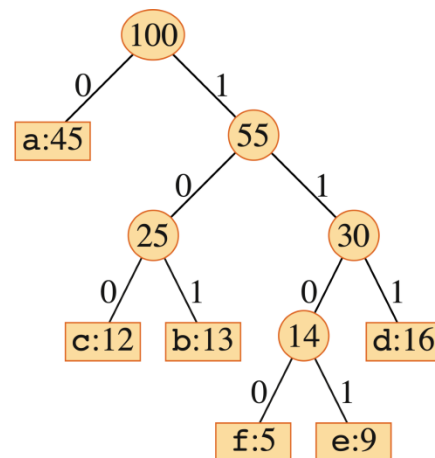
Huffman codes

□ Binary tree representation

Use a binary tree whose leaves are the characters. The codeword for a character is given by the simple path from the root down to that character's leaf, where going left is 0 and going right is 1.



(a)



(b)

The trees using (a) the fixed-length code and (b) the optimal prefix-free code.

- Here, each leaf has its character and frequency (in thousands). Each internal node holds the sum of the frequencies of the leaves in its subtree. 6

Huffman codes

- An optimal code is always given by a *fully* binary tree: each internal node has two children $\Rightarrow$ if C is the alphabet for the characters, then the tree has $|C|$ leaves and $|C| - 1$ internal nodes.
- **How to compute the number of bits to encode a file for alphabet C given tree T :**
- For each character $c \in C$, denote its frequency by $c.freq$. Denote the depth of c in T by $d_T(c)$, which equals the length of c 's codeword. Then the number of bits to encode the file, the *cost* of T , is

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c)$$

Constructing a Huffman code

- Huffman invented a greedy algorithm that constructs an optimal prefix-free code, called a *Huffman code* in his honor.
- The algorithm builds tree T bottom-up. It repeatedly selects two nodes with the lowest frequency and makes them children of a new node whose frequency is the sum of the two nodes' frequencies. It uses a min-priority queue Q keyed on the *freq* attribute, which all nodes have.

HUFFMAN(C)

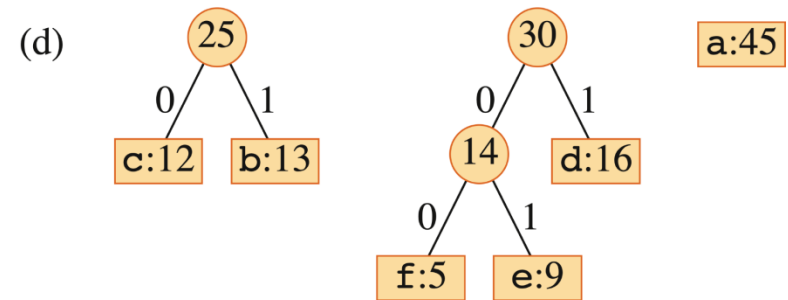
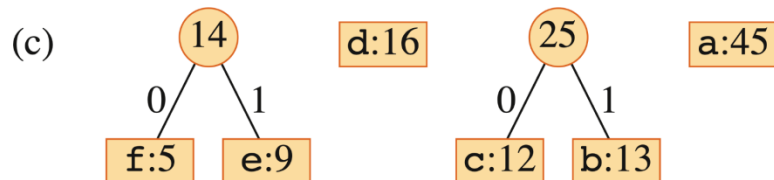
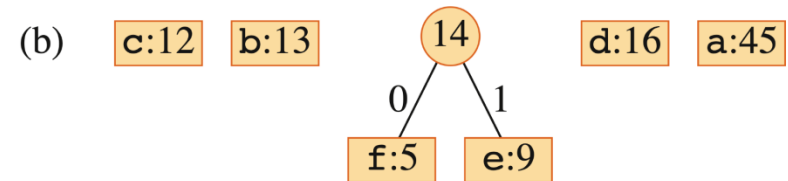
```
1   $n = |C|$ 
2   $Q = C$ 
3  for  $i = 1$  to  $n - 1$ 
4      allocate a new node  $z$ 
5       $x = \text{EXTRACT-MIN}(Q)$ 
6       $y = \text{EXTRACT-MIN}(Q)$ 
7       $z.\text{left} = x$ 
8       $z.\text{right} = y$ 
9       $z.\text{freq} = x.\text{freq} + y.\text{freq}$ 
10      $\text{INSERT}(Q, z)$ 
11 return  $\text{EXTRACT-MIN}(Q)$     // the root of the tree is the only node left
```


Constructing a Huffman code: Example

- Build an optimal prefix-free tree using the following frequencies:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5

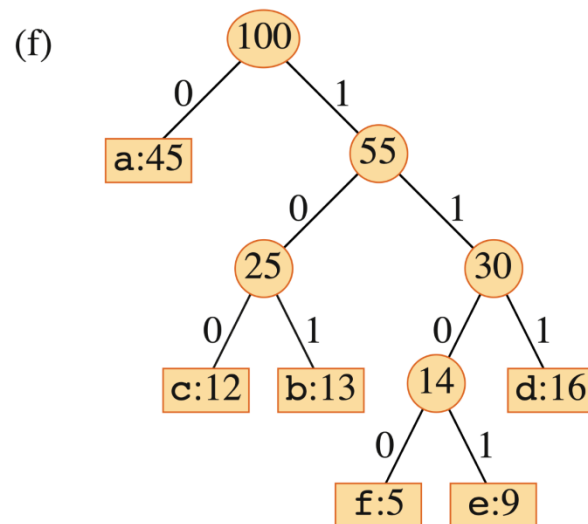
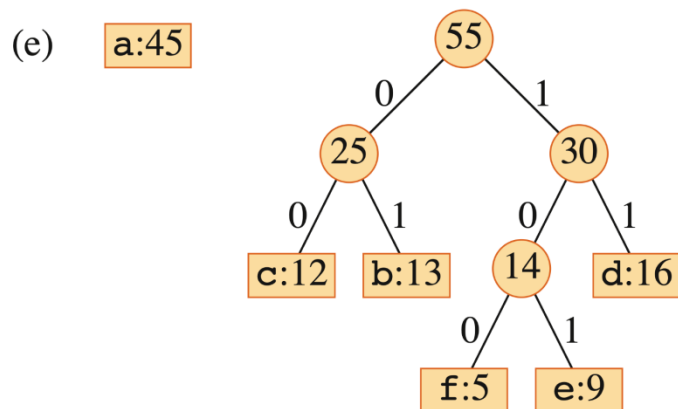
(a) f:5 e:9 c:12 b:13 d:16 a:45



Constructing a Huffman code: Example

- Build an optimal prefix-free tree using the following frequencies:

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5



- Running time:** Let $n = |C|$. The running time depends on how the min-priority queue Q is implemented. If with a binary min-heap, can initialize Q in $O(n)$ time. The **for** loop runs $n - 1$ times, and each INSERT and EXTRACT-MIN call takes $O(\lg n)$ time $\Rightarrow O(n \lg n)$ time in all.

Reading

- Section 15.3.